

FINAL PROJECT REPORT

TRACE-X

TRACE THE THREAT. PLAN THE RESPONSE.

Intelligent Cyber Incident Response & Containment

Problem 92 — Cyber Incident Response Planner

NAME:

M. SREEJA
V. PRASANNA
S. RENUKA
M. SAHASRA



1 Abstract & Introduction

TRACE-X is a DAA-focused Cyber Incident Response Planner designed to convert incoming cyber incidents into a structured and explainable response plan. The system combines a Priority Queue, Breadth-First Search (BFS), and Dijkstra's Shortest Path algorithm to support incident prioritization, network impact analysis, and containment-route planning.

Problem Background

During a cyber incident, security teams may receive multiple alerts at the same time. Detecting an alert is only the first step. The responder must determine what needs attention first, which connected systems may be affected, and how the incident can be contained efficiently.

Proposed Solution

TRACE-X models the affected infrastructure as a graph and uses DAA algorithms to make each response step explainable. Incidents are prioritized, reachable systems are explored, and an efficient containment route is calculated.

Purpose of the Project

- Provide a structured incident-response workflow.
- Apply core DAA algorithms to a practical cybersecurity problem.
- Present decisions through a clear dashboard and response plan.

2 Problem Statement, Objectives & Scope

Problem Statement. Design a Cyber Incident Response Planner that prioritizes multiple incidents, identifies potentially affected systems in a connected network, and determines an efficient containment route using Priority Queue, BFS, and Dijkstra's Shortest Path.

Objectives

- Prioritize incidents using severity, impact, and critical-asset factors.
- Trace reachable systems from a compromised node using BFS.
- Calculate a minimum-cost / minimum-distance containment route using Dijkstra.
- Generate an explainable response plan for the responder.
- Provide a clean interface for viewing incidents and network analysis.

Scope

- Incident intake and prioritization.
- Graph-based network analysis.
- Blast-radius identification.
- Containment-route planning.
- Dashboard visualization and response recommendation.

Target Users

Security students, cybersecurity teams, SOC-style responders, and organizations seeking a structured incident-response planning workflow.

3 Features & Functional Requirements

Feature	Functional Requirement
Incident Dashboard	View incidents with Critical / High / Medium priority.
Incident Prioritization	Calculate and organize incidents using a priority score.
Network Graph	Represent systems and connections as graph nodes and edges.
BFS Analysis	Trace connected/reachable systems and identify potential blast radius.
Dijkstra Analysis	Calculate an efficient containment path.
Incident Reconstruction	Connect entry point, spread, critical asset, priority, and containment.
Response Plan	Display an explainable next-action sequence.
Validation	Validate required incident and network inputs.

System Requirements

- **Frontend:** React
- **Backend:** Python + Flask
- **Runtime:** Python environment and modern web browser
- **Data:** incident records and network graph data

4 Algorithms & Methodology

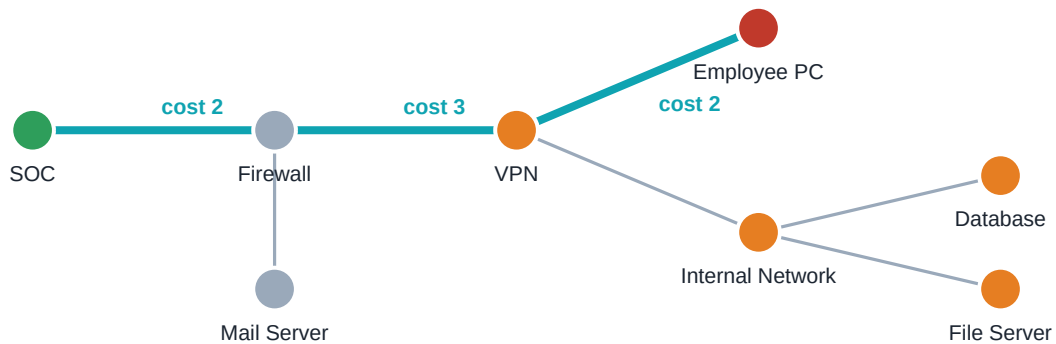
System Workflow



Algorithms Used

Algorithm	Decision	Role in TRACE-X
Priority Queue	WHAT FIRST?	Orders incidents according to their priority.
BFS	WHAT IS AT RISK?	Explores reachable systems from the compromised node.
Dijkstra	HOW TO CONTAIN?	Finds a minimum-cost / minimum-distance path.

Network Model (illustrative example)



● Safe base (SOC)

● Compromised

● Blast radius

● Containment route

Illustrative graph: if Employee PC is compromised, BFS traces VPN, Internal Network, Database and File Server as the blast radius; Dijkstra finds the cheapest route from the SOC (SOC, Firewall, VPN, Employee PC; total cost 7).

Incident Reconstruction Method

- Identify the likely compromised entry point.
- Use BFS to trace connected systems and estimate the affected area.
- Identify critical assets within the reachable area.
- Use Priority Queue to determine response urgency.
- Use Dijkstra to calculate an efficient containment route.
- Generate a traceable response plan.

5 Technologies & Implementation

Layer	Technology / Implementation
Frontend	React — dashboard, incident views, graph visualizations, response-plan UI
Backend	Python + Flask — API endpoints, incident processing, graph management
DAA Engine	Priority Queue, BFS, Dijkstra
Data Model	Incident records + network graph nodes and edges
Tools	VS Code / Python environment / browser
API Flow	React frontend > Flask API > DAA engine > response data

Architecture



Implementation Details

- The frontend sends incident and network information to the Flask backend.
- The backend processes the graph and invokes the required DAA algorithm.
- The algorithm output is returned to the frontend as structured response information.
- The dashboard presents the priority, affected systems, route, and response sequence.

6 Results, Testing & Performance

Project Results

- Incident records can be organized by priority.
- The network can be represented as connected graph data.
- BFS identifies reachable systems from a selected compromised node.
- Dijkstra calculates an efficient path based on assigned edge costs.
- The system combines outputs into an explainable response workflow.

Testing Methods

ID	Input / Scenario	Expected Result	Status
TC-01	Multiple incidents	Incidents ordered by priority	To verify
TC-02	Compromised node + graph	Reachable systems identified	To verify
TC-03	Weighted network	Minimum-cost route returned	To verify
TC-04	Invalid/missing input	Validation message displayed	To verify
TC-05	End-to-end incident	Response plan generated	To verify

Performance Analysis

Performance should be reported using measured test results. No unmeasured real-world performance claims are included in this report.

Screenshots / Output

[Insert dashboard, network-analysis and containment screenshots here once the application is running.]

7 Conclusion, Future Scope & References

Conclusion

TRACE-X demonstrates how fundamental DAA concepts can be applied to a practical cybersecurity response problem. The combination of Priority Queue, BFS, and Dijkstra creates a clear pipeline from incident prioritization to network-risk analysis and containment planning.

Future Scope

- Real-time security-log ingestion.
- Integration with SIEM/security monitoring platforms.
- Role-based authentication and access control.
- More detailed graph-based attack-path analysis.
- Historical incident storage and incident reports.
- Automated response playbooks and controlled remediation actions.

References / Documentation

- Python Documentation — <https://docs.python.org/>
- Flask Documentation — <https://flask.palletsprojects.com/>
- React Documentation — <https://react.dev/>
- Dijkstra's algorithm and graph algorithms — standard algorithm references / course material.
- BFS and Priority Queue — standard Data Structures & Algorithms references / course material.